Foreign Whispers

An Open-Source End-to-End AI Video Dubbing Pipeline

Advait Jishnani (aj4700) · Tilak Bhansali (tb3525)

AI Online | Professor Pantelis Monogioudis

April 2026

1. Overview

Foreign Whispers is an end-to-end, open-source video dubbing pipeline. Given a YouTube URL, it automatically downloads the video, transcribes and diarizes the audio, translates each segment into Spanish, synthesizes a gender-aware dubbed audio track with temporal alignment, and stitches the result back into the original video ,all accessible from a browser-based Dubbing Studio UI. The entire pipeline runs on commodity CPU hardware with no GPU, API key, or paid service required.

The central challenge is not simply translating text ,it is ensuring synthesized speech fits the original timing. A Spanish translation of an English sentence is typically 20–30% longer in syllable count. Without compensation, the dubbed audio continuously drifts, producing a video where speech and lip movement diverge. This report describes the architecture, key innovations, implementation challenges, and results.

This project began as an individual effort by Advait Jishnani and grew into a collaboration with Tilak Bhansali, who extended the pipeline with speaker diarization, gender-aware TTS, neural translation reranking, and a significantly improved deployment and alignment architecture.

2. System Architecture

The application uses a distributed, container-based architecture with clear separation of concerns. The cpu profile (used throughout this project) starts two containers:

Container	Tool	Port	Role
foreign-whispers-api	FastAPI (CPU)	8080	Pipeline orchestrator; runs Whisper base + Edge TTS in-process
foreign-whispers-frontend	Next.js + shadcn/ui	8501	Dubbing Studio UI; proxies all requests to API

An optional nvidia profile adds two GPU-only containers (Whisper STT on port 8000 and Chatterbox TTS on port 8020), but these were not used in our testing. The pipeline runs fully on CPU using Whisper base for transcription and Edge TTS for voice synthesis.

2.1 Frontend Dubbing Studio

Built with Next.js 14 and shadcn/ui, the frontend presents a pipeline tracker that visualizes each stage (Download → Transcribe → Diarize → Translate → Synthesize → Stitch). Each stage button triggers a REST call to the API and displays progress, errors, and results inline. The video player renders the dubbed output with synchronized WebVTT captions through the browser's native track element no subtitle burn-in required.

2.2 FastAPI Backend

The API container is CPU-only and follows a layered architecture: thin HTTP routers in api/src/routers/, HTTP-agnostic business logic in api/src/services/, and Pydantic request/response models in api/src/schemas/. All source directories are bind-mounted into the container so edits on the host are visible without rebuilds. Models are loaded lazily only on the first actual request keeping API startup fast.

2.3 Data Flow

YouTube URL → yt-dlp download (MP4 + closed captions) → ffmpeg audio extraction (WAV) → Whisper base transcription (timestamped JSON) → pyannote diarization (speaker labels injected) → argostranslate + MarianMT reranking (EN→ES JSON) → Edge TTS synthesis (per-segment WAV, time-stretched) → ffmpeg remux (-c:v copy, no re-encode) → Dubbed MP4 + WebVTT captions

3. Pipeline Stages

The pipeline consists of six sequential stages. Each stage reads from and writes to the shared pipeline_data/ directory, enabling caching and incremental re-runs.

P1: Download

The video and its captions are fetched from YouTube using yt-dlp. The unique video ID is extracted, the MP4 downloaded, and auto-generated captions retrieved in JSON format. All files are stored under pipeline_data/api/videos/ and pipeline_data/api/youtube_captions/.

P2: Transcribe

Whisper base runs inside the API container on CPU, producing a word-level transcript with precise timestamps for each segment. The base model was chosen for CPU performance transcription of a 10-minute video takes approximately 90 seconds. The output is saved to pipeline_data/api/transcriptions/whisper/.

P3: Diarize

Speaker diarization is performed using pyannote.audio, which detects speaker turn boundaries and assigns stable speaker IDs (SPEAKER_00, SPEAKER_01, etc.) to each segment. The diarization output is injected directly into the transcript JSON, enriching each segment with a speaker label. This enables gender-aware voice assignment in the TTS stage. A HuggingFace token and model access agreement are required.

P4: Translate

Each transcript segment is translated from English to Spanish using argostranslate (offline OpenNMT). A neural reranking step using MarianMT (Helsinki-NLP/opus-mt-en-es) generates 5 beam-search candidates and scores each by predicted TTS duration using a syllable-rate heuristic.

Candidates that fit within the target time window are returned shortest-first. A word-level fallback (truncate_for_duration_budget) drops trailing words at sentence boundaries as a last resort. Results are saved to pipeline_data/api/translations/argos/.

P5: Text-to-Speech (TTS)

The TTS stage synthesizes spoken audio from translated segments. The engine is selected automatically at startup:

- Edge TTS (default) Microsoft neural voices, free, no GPU required. Male speakers receive es-ES-AlvaroNeural; female speakers receive es-ES-ElviraNeural, determined by pyannote speaker index parity.
- Coqui Tacotron2 fully offline fallback for air-gapped environments.
- Chatterbox GPU voice cloning; only available on the nvidia profile, not used in this project.

The edge-tts library is fully async, but synthesis is dispatched from a ThreadPoolExecutor. To avoid RuntimeError when a loop is already running, EdgeTTSClient.tts_to_file() detects the running loop and dispatches to a fresh thread where asyncio.run() is safe. Edge TTS produces MP3, which is transcoded to WAV via pydub before the rest of the pipeline processes it.

When alignment is enabled (FW_ALIGNMENT=on, default), each synthesized segment is time-stretched using pyrubberband (a phase-vocoder, pitch-preserving time stretcher), bounded between 0.75× and 1.25× speed. Segments shorter than 50% of the target window play at natural speed with silence padding.

P6: Stitch

The stitch stage combines the dubbed audio WAV and the original video using ffmpeg with direct stream copy (-c:v copy), avoiding re-encoding entirely for speed and quality preservation. WebVTT captions are written alongside the dubbed MP4 for accessibility. The final file is saved to pipeline_data/api/dubbed_videos/.

4. Key Technical Innovations

4.1 Neural Translation Reranking

The naive approach translate once and use the result produces translations that are often too long for their time slot. We implemented a dual-backend reranking pipeline in foreign_whispers/reranking.py: argostranslate generates a baseline translation; MarianMT generates 5 alternatives via beam search; each candidate is scored by predicted TTS duration; candidates fitting within the target time window are returned shortest-first. A word-level truncation fallback drops words at sentence boundaries rather than mid-word, preventing garbled speech.

4.2 Ridge Regression Duration Prediction

To predict how long a Spanish TTS segment will take before synthesizing it (enabling smarter scheduling), a Ridge regression model was trained on paired (text_features, actual_tts_duration) examples. Features: character count, syllable count (Spanish-aware), word count. The model is stored as tts_duration_ridge.json and loaded in foreign_whispers/alignment.py. Predictions achieve approximately 85ms median absolute error, compared to ~320ms for the naive syllable-rate heuristic.

4.3 Global Alignment with Dynamic Programming

Rather than processing each segment independently, a global alignment pass (global_align_dp in alignment.py) computes per-segment stretch factors, tags each with an AlignAction (MILD_STRETCH, STRETCH, REQUEST_SHORTER, or PAD), and uses DP beam search to minimize the maximum stretch factor across all segments simultaneously distributing slack from easy segments to hard ones. A sidecar .align.json report records per-segment stretch factors, raw durations, and actions for evaluation.

4.4 Gender-Aware Speaker Voice Mapping

pyannote.audio assigns stable speaker labels. The implementation maps these to gendered voices round-robin: even-indexed speakers receive the male Edge TTS voice (es-ES-AlvaroNeural); odd-indexed speakers receive the female voice (es-ES-ElviraNeural). Explicit gender suffixes in speaker labels (_F, FEM, etc.) take precedence over the parity fallback. This ensures men sound like men and women sound like women throughout the dubbed video.

4.5 TTS Engine Fallback Chain

Rather than hard-coding a single TTS backend, an engine factory implements a prioritized fallback: Chatterbox GPU (if reachable) → Edge TTS (default, free, gender-aware, excellent quality) → Coqui Tacotron2 (offline, air-gapped). In practice, Edge TTS was the engine used throughout this project, requiring no GPU or API key.

5. Implementation Challenges

5.1 Spanish Syllable Expansion

English is stress-timed with many reduced vowels; Spanish is syllable-timed and retains all vowel sounds. An average English sentence spoken in 3 seconds takes approximately 3.6–4.2 seconds in Spanish. Without proactive duration management, drift accumulates a 10-minute video can end up 2+ minutes longer in Spanish. The solution is a multi-stage pipeline: predict duration before synthesis → select shorter translations → time-stretch within quality bounds → pad or trim to target window.

5.2 Async Edge TTS in a Synchronous Pipeline

The edge-tts library is fully async, but the TTS pipeline dispatches synthesis calls from a ThreadPoolExecutor using synchronous wrappers. Calling asyncio.run() from inside a thread that already has a running event loop raises RuntimeError. The fix: EdgeTTSClient.tts_to_file() detects whether a loop is running and, if so, dispatches the coroutine to a fresh thread where asyncio.run() is safe. Edge TTS output (MP3) is transcoded to WAV via pydub, keeping the rest of the pipeline format-agnostic.

5.3 Lazy Model Loading

Heavyweight models (pyannote, Coqui) initialize in 30–60 seconds if loaded at import time, stalling every API restart during development. The solution: a lazy singleton pattern via _get_tts_engine() loads the model only on the first actual TTS request, keeping API startup fast.

5.4 Test Suite Migration

Extending the reference implementation introduced breaking API changes: removal of a module-level tts singleton in favor of a lazy factory, signature changes to text_file_to_speech() and _synced_segment_audio(), and updated return types. All 4 test files were updated; 23/23 targeted tests now pass. 13 remaining failures are pre-existing baseline issues unrelated to these changes.

6. Results

6.1 Videos Dubbed

The pipeline was run end-to-end on all four YouTube videos from the 60 Minutes library:

- Strait of Hormuz disruption threatens to shake global economy (GYQ5yGV_-Oc) 6 min 45 sec, 170 segments, 1,118 words translated
- Alysa Liu: The 60 Minutes Interview (6O3HLPWatuU) 12 min 59 sec etc.

All of them were successfully dubbed into Spanish with WebVTT subtitle overlays. The Dubbing Studio UI correctly displayed pipeline stage statuses, allowed switching between original and dubbed video playback, and showed segment count, speech duration, and word count statistics.

6.2 Alignment Metrics

From the .align.json sidecar reports on the Strait of Hormuz test video:

Metric	Value
Mean absolute duration error (with alignment)	~180ms
Mean absolute duration error (without alignment)	~640ms
Segments requiring reranking (>25% stretch needed)	~18%
Segments using silence padding (TTS shorter than target)	~22%
Severe stretch events (>25%)	<5%

6.3 Pipeline Stage Latencies (CPU, ~10 min video)

Stage	Latency
Download	~15s
Transcribe (Whisper base, CPU)	~90s
Translate + Rerank	~45s
TTS Synthesis (Edge TTS, 3 workers)	~120s
Stitch (ffmpeg -c:v copy)	~5s
Total	~275s (~4.5 min)

7. Limitations and Future Work

- Global optimization The current alignment uses a greedy left-to-right pass with local look-ahead. A full Integer Linear Program (ILP) formulation would allow globally optimal allocation of slack time across segments.
- Prosody transfer Time-stretching preserves naturalness only within [0.75×, 1.25×]. A neural vocoder (e.g., VITS) could synthesize directly at a target duration without post-hoc stretching.
- Lip-sync The current approach makes no attempt at lip-sync beyond temporal alignment. Future work could apply a video translation model (e.g., Wav2Lip) to animate speaker mouths to match the dubbed audio.
- Multi-language support The pipeline is hardcoded to English→Spanish. Generalizing requires swapping argostranslate models and TTS voice selections, which is architecturally straightforward but untested.
- Streaming synthesis Currently all TTS segments are synthesized before assembly. A streaming architecture would start playing early segments while later segments are still synthesizing, reducing perceived latency.

8. Conclusion

Foreign Whispers successfully implements a fully automated, browser-operated video dubbing pipeline running on commodity CPU hardware without proprietary APIs, GPU access, or cloud services. Starting from an initial individual implementation, the collaboration between Advait Jishnani and Tilak Bhansali produced a system with five key contributions over the baseline:

- Neural reranking eliminates robotic rushed speech from over-long translations
- Ridge regression duration prediction enables smarter alignment planning before synthesis
- Gender-aware TTS via pyannote diarization satisfies the perceptual requirement that speakers sound gender-appropriate in the dubbed output
- Global DP alignment minimizes drift accumulation across long videos
- Fault-tolerant TTS fallback chain makes the pipeline work on any machine with internet access

The result is a system that can take a 60 Minutes interview in English and produce a watchable Spanish dub in under 5 minutes on a MacBook no GPU required.

The complete codebase is available at https://github.com/tilak30/foreign-whispers.